

Examen final de Programació i Estructures de Dades (PRD)

Data: 15 de juny de 2023. Duració: 2h 30min.

Publicació de notes provisionals: 21/06/2023 (Atenea)

Revisió de l'examen: 22/06/2023 – 10:30h, Sala D6-221 (edifici D6, 2ª planta)

IMPORTANT: cal resoldre i lliurar els problemes 1, 2 y 3 en fulles separades

PROBLEMA 1: 4.0 punts, temps de resolució: 50 min.

Es desitja implementar un programa per a gestionar les comandes dels repartiments d'una empresa de venda online. Aquestes comandes s'organitzen seguint un esquema de cua modificat lleugerament. Aquesta modificació consisteix en que, en cas d'existir ja una comanda per a un cert client a la cua, les comandes d'aquest client s'agrupen per tal de facilitar així els lliuraments, tant pel client com pel repartidor. En cas que no hi hagi cap comanda prèvia per a un determinat client, la cua actuarà normalment, afegint la comanda al final d'aquesta (disciplina FIFO). El disseny resultant d'aquest programa contempla la definició de les constants simbòliques i estructures de dades següents:

```
#define MAX_C 50

#define OK      0
#define ERROR -1

typedef struct {
    int num;
    char lletra;
}t_nif;

typedef struct {
    t_nif client;
    int producte;
    int hora_max_lliurament; /* Format HHMM */
    int data_lliurament;    /* Format AAAAMMDD */
}t_comanda;

typedef struct {
    t_comanda comandes[MAX_C];
    int ncomandes;
}t_repartiment;
```

El repartidor pot tenir a la cua del repartiment fins a MAX_C comandes. Per simplificar-ho, cada comanda conté el NIF del client (número i lletra), l'identificador del producte associat a la comanda (un únic producte), així com l'hora màxima i data de lliurament.

A partir d'aquesta descripció inicial, cal implementar les funcions següents:

- (0.75 punts)** void mostrar_repartiment (t_repartiment r); aquesta funció ha de mostrar les dades de la cua del repartiment rebut com a paràmetre. Per fer-ho, heu d'utilitzar la funció void mostrar_comanda (t_comanda c); que també cal implementar i que ha de mostrar la informació d'una comanda, tal i com detallen els exemples d'execució inclosos al final de l'enunciat d'aquest problema.


```

printf("Id producte:");
scanf ("%d", &c.producte);

printf("Indica hora maxima de lliurament:");
scanf ("%d:%d%c", &hora, &minuts);
while (!(hora>=0 && hora<=23) && (minuts >=0 && minuts<=59)))
{
/* També és correcte (hora < 0 || hora >23 || minuts < 0 || minuts > 59) */
printf("Error a l'hora de lliurament. Introdueix nova hora:");
scanf ("%d:%d%c", &hora, &minuts);
}
c.hora_max_lliurament = hora*100 + minuts;

printf("Indica data de lliurament:");
scanf ("%d/%d/%d%c", &dia, &mes, &any);
c.data_lliurament = any * 10000 + mes * 100 + dia;

return (c);
}

```

- c. **(1.25 punts)** int encuar_comanda (t_repartiment *r, t_comanda c); aquesta funció ha d'afegir la comanda c rebuda com a paràmetre a la cua del repartiment apuntat pel paràmetre r, tenint en compte els següents aspectes. Si el repartiment té ja el nombre màxim de comandes a la cua, cal que la funció retorni directament ERROR. Si el repartiment ja tenia almenys una comanda per a aquest client a la cua, la nova comanda s'ha d'inserir juntament amb les seves comandes anteriors, desplaçant la resta de comandes de la cua en cas necessari. Els camps hora_max_lliurament i data_lliurament, caldrà modificar-los per tal que siguin iguals als de les comandes que ja eren prèviament a la cua del repartiment. Si el client no tenia cap comanda a la cua, la comanda s'afegeix al final d'aquesta. Finalment, cal que la funció retorni OK.

```

int encuar_comanda (t_repartiment *r, t_comanda c)
{
    int i = 0, pos = 0, trobat = 0;

    if (r->ncomandes == MAX_C)
        return (ERROR);

    while (!trobat && i < r->ncomandes)
    {
        if (r->comandes[i].client.num == c.client.num)
            trobat = 1;
        else
            i++;
    }

    if (trobat)
    {
        pos = i;
        c.hora_max_lliurament = r->comandes[pos].hora_max_lliurament;
        c.data_lliurament = r->comandes[pos].data_lliurament;

        for (i = r->ncomandes; i > pos; i--)
            r->comandes [i] = r->comandes [i-1];

        r->comandes[pos] = c;
    }
}

```

```

else
    r->comandes[r->ncomandes] = c;

r->ncomandes++;
return (OK);
}

```

- d. **(1 punt)** `int eliminar_comanda_client (t_repartiment *r, t_nif client);` aquesta funció ha d'eliminar de la cua del repartiment apuntat pel paràmetre `r` totes les comandes del client amb NIF igual al paràmetre `client`. Heu de controlar que hi hagi comandes del client a la cua. Si no hi té comandes, la funció ha de retornar `0`. Si el client hi té comandes, ha de retornar el nombre de comandes eliminades.

```

int eliminar_comanda_client (t_repartiment *r, t_nif client)
{
    int i, j;
    int nelim = 0;

    if (r->ncomandes > 0)
    {
        for (i = 0; i < r->ncomandes; i++)
        {
            if (r->comandes[i].client.num == client.num)
            {
                nelim++;
                for (j = i; j < r->ncomandes - 1; j++)
                {
                    r->comandes[j] = r->comandes[j+1];
                }
                i--; /* Per si el client té més d'alguna comanda, tenir en
                     compte la posició que s'acaba d'esborrar */
                r->ncomandes--;
            }
        }
    }

    return (nelim);
}

```

Exemple de sortida del programa (en negreta: dades que introdueix l'usuari):

La cua de comandes del repartiment es buida.

Llegint comanda:
Nif usuari: **11111111-5**
Error al nif del client. Introdueix nou nif: **11111111-A**
Id producte: **554**
Indica hora maxima lliurament: **25:44**
Error a hora maxima lliurament. Introdueix nova hora: **22:00**
Indica data lliurament: **15/06/2023**

Comanda inserida correctament.

La cua de comandes del repartiment te 1 comanda.
Nif: 11111111-A
Id producte: 554
Hora maxima lliurament: 22:00
Data lliurament: 15/06/2023

Introdueix nif de client: **12345678-H**
El client 12345678-H no te cap comanda.

Introdueix nif de client: **11111111-A**
S'ha eliminat 1 comanda del client 11111111-A.

La cua de comandes del repartiment es buida.

PROBLEMA 2: 4.5 punts, temps de resolució: 75 min.

A continuació, es detallen les estructures de dades necessàries per a la implementació d'una llista enllaçada simple, capaç d'emmagatzemar una seqüència de valors (int):

```
typedef struct node
{
    int valor;           /* Valor guardat al node de la llista */
    struct node *next;   /* Punter al següent node de la llista */
}t_node;
```

```
typedef struct
{
    int size;           /* Nombre de valors guardats */
    t_node *head;       /* Punter al primer node de la llista */
}t_llista;
```

A partir de les estructures de dades definides, es demana la implementació de les següents funcions:

- a. **(1 punt)** int comparar (t_llista *llista1, t_llista *llista2); aquesta funció cal que compari el contingut de les llistes enllaçades apuntades pels paràmetres llista1 i llista2. En cas que el contingut d'ambdues llistes enllaçades sigui idèntic, la funció cal que retorni 1. En canvi, si el contingut de les llistes difereix, la funció cal que retorni 0.

Nota: es considera que el contingut de les dues llistes és idèntic quan aquestes guarden exactament el mateix valor en cadascuna de les seves posicions.

```
int comparar (t_llista *llista1, t_llista *llista2)
{
    t_node *tmp1, *tmp2;

    if (llista1->size != llista2->size)
        return (0);

    tmp1 = llista1->head;
    tmp2 = llista2->head;

    while (tmp1 != NULL)
    {
        if (tmp1->valor != tmp2->valor)
            return (0);

        tmp1 = tmp1->next;
        tmp2 = tmp2->next;
    }

    return (1);
}
```

- b. **(1 punt)** int identificar_ordre (t_llista *llista); aquesta funció cal que identifiqui si els valors guardats a la llista enllaçada apuntada pel paràmetre llista estan ordenats de forma creixent o decreixent. En cas que els valors guardats a la llista mantinguin un ordre creixent entre ells, la funció cal que retorni 1. En canvi, si els valors guardats a la llista mantenen un ordre decreixent, cal que retorni -1. Si no existeix però cap ordre entre els valors guardats, la funció cal que retorni 0.

Nota: Si la llista és buida o conté només un sol valor, s'assumeix que NO està ordenada. A més, si conté tots els seus valors idèntics, s'assumeix que manté un ordre creixent. Per exemple, si la mida d'una llista enllaçada és 4 i el seu contingut és [1, 1, 1, 1], s'assumiria que aquesta està ordenada de forma creixent.

```
int identificar_ordre (t_llista *llista)
{
    t_node *actual, *prev;
    int asc = 1, des = 1;

    if (llista->size < 2)
        return (0);

    prev = llista->head;
    actual = prev->next;

    while (actual != NULL && (asc || des))
    {
        if (actual->valor < prev->valor)
            asc = 0;
        else if (actual->valor > prev->valor)
            des = 0;

        prev = actual;
        actual = actual->next;
    }

    if (asc)
        return (1);
    else if (des)
        return (-1);
    else
        return (0);
}
```

- c. **(1 punt)** void obtenir_array (t_llista *llista, int **v); aquesta funció cal que creï un nou array dinàmic d'enters, inicialitzat amb els mateixos valors que guarda la llista enllaçada apuntada pel paràmetre llista (respectant també les seves posicions). La funció caldrà que retorni un punter al nou array dinàmic a través del paràmetre v. En cas de succeir qualsevol problema durant la reserva de la memòria dinàmica necessària pel nou array, la funció cal que retorni NULL a través del paràmetre v esmentat anteriorment.

```
void obtenir_array (t_llista *llista, int **v)
{
    t_node *tmp;
    int i = 0;

    *v = (int *)calloc(llista->size, sizeof(int));

    if (*v != NULL)
    {
        tmp = llista->head;

        while (tmp != NULL)
        {
            (*v)[i] = tmp->valor;
        }
    }
}
```

```

        tmp = tmp->next;
        i++;
    }
}
}

```

- d. **(1.5 punts)** `t_llista * duplicar (t_llista *llista);` aquesta funció cal que creï una nova llista enllaçada que sigui una còpia exacta de la llista enllaçada apuntada pel paràmetre `llista`, amb un contingut idèntic. És important remarcar que les dues llistes (original i còpia) hauran d'acabar sent totalment independents (els canvis successius en qualsevol d'aquestes llistes no hauran d'afectar de cap forma a l'altra). La funció ha d'acabar retornant un punter a la nova llista còpia, o `NULL` en cas de succeir qualsevol problema durant la reserva de memòria dinàmica necessària per a la seva creació.

Nota: En cas de succeir qualsevol error durant la reserva de memòria dinàmica, la funció ha d'assegurar-se d'alliberar tots els blocs de memòria dinàmica prèviament reservats per a l'operació, abans d'acabar retornant `NULL`.

```

t_llista * duplicar (t_llista *llista)
{
    t_node *tmp, *node, *tail;
    int error = 0;

    t_llista *dup = (t_llista *)malloc(sizeof(t_llista));
    if (dup == NULL)
        return (NULL);

    dup->size = 0;
    dup->head = NULL;
    tmp = llista->head;
    while (tmp != NULL && !error)
    {
        node = (t_node *)malloc(sizeof(t_node));
        if (node == NULL)
            error = 1;
        else
        {
            node->valor = tmp->valor;
            node->next = NULL;

            if (dup->head == NULL)
                dup->head = node;
            else
                tail->next = node;

            dup->size++;
            tail = node;
            tmp = tmp->next;
        }
    }

    if (error == 1)
    {
        while (dup->head != NULL)
        {
            tmp = dup->head->next;
            free(dup->head);
            dup->head = tmp;
        }
    }

    return dup;
}

```

```

        free (dup->head);
        dup->head = tmp;
    }

    free (dup);
    return (NULL);
}

return (dup);
}

```

PROBLEMA 3: 1.5 punts, temps de resolució: 25 min.

Implementa la funció:

```
void encreuar_meitats (unsigned int *s1, unsigned int *s2);
```

Aquesta funció rep dos punters (s1 i s2) a variables enteres sense signe (unsigned int), cadascuna de les quals guarda una seqüència de 32 bits. La funció ha de ser capaç d'encreuar les meitats (porcions de 16 bits) de les seqüències de bits que guarden les variables apuntades pels paràmetres s1 i s2.

Per exemple, imaginem que les variables apuntades pels paràmetres s1 i s2 guarden les seqüències de bits identificades pels valors hexadecimals 0x88880000 i 0xFFFF1111, respectivament:

```

s1 → 1000 1000 1000 1000 | 0000 0000 0000 0000
s2 → 1111 1111 1111 1111 | 0001 0001 0001 0001

```

Un cop executada la funció, aquestes variables apuntades han d'acabar guardant les seqüències de bits identificades pels valors hexadecimals 0x1111FFFF i 0x8888:

```

s1 → 0001 0001 0001 0001 | 1111 1111 1111 1111
s2 → 0000 0000 0000 0000 | 1000 1000 1000 1000

```

Nota: cal aconseguir la funcionalitat desitjada utilitzant exclusivament operadors a nivell de bit. No s'acceptaran respostes que utilitzin operadors aritmètics per tal d'aconseguir el mateix propòsit.

```

void encreuar_meitats (unsigned int *s1, unsigned int *s2)
{
    unsigned int aux1, aux2, aux3, aux4;

    aux1 = *s2 << 16;
    aux2 = *s2 >> 16;

    aux3 = *s1 << 16;
    aux4 = *s1 >> 16;

    *s1 = aux1 | aux2;
    *s2 = aux3 | aux4;
}

```